Common Core: Phase 3 Physical Design

Lab P3: Area, Timing and Scaling of Larger Blocks

Parameter Sweeps, Memories and PPA Trade-offs

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Automate synthesis and timing over parameter sets, measure how the area and timing of `arb_mem` and `dcache` scale, find where the area goes and why the cache hit path is critical, and argue when flip-flop storage must give way to SRAM macros.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Labs P2, R3 and R4 (verified in V2 and V3)
Designs	`arb_mem` (R3), `dcache` (R4); RTL unchanged except an optional memory-wrapper refactor
Reused in	T-AM1 (area budget for the cache recommendation), T-RTL2 (STA of the CDC top)

Contents

1 Overview **3**

2 How Storage-Dominated Blocks Scale **3**

3 Parameter Sweeps **4**

3.1 `syn/configs.txt` 4

3.2 `sweep.sh` and the `sweep` Target 4

4 `arb_mem`: Memory Built from Flip-Flops **6**

5 `dcache`: Where the Area Goes **7**

5.1 Model First 7

5.2 Measuring the Breakdown 7

6 `dcache`: the Critical Path Through the Tag Compare **8**

7 SRAM Macros versus Flip-Flops **9**

7.1 How a Synchronous Read Changes the Controller 9

8 Cache Geometry Sweep: `NUM_LINES` and `LINE_WORDS` **10**

8.1 Stretch: 2-Way Set-Associative versus Direct-Mapped 10

9 The PPA Report **10**

10 Further Reading: Toward Place and Route **11**

11 Requirements / Tasks **11**

12 Repository and Flow **12**

13 Deliverables **12**

14 Assessment **13**

15 Review Questions **13**

16 References **14**

1 Overview

Power, performance and area (PPA) are the three numbers every architecture decision is eventually judged by. A cache that doubles the hit rate is worthless if it does not fit the area budget or limits the clock of the whole core. Architects and RTL designers therefore need a quick, repeatable way to answer "what does this parameter cost?" long before a floorplan exists.

In Labs P1 and P2 you synthesized and timed single configurations of small blocks. This lab scales up in two directions: *automation* (a `sweep` target that runs synthesis and STA over a list of parameter sets and writes a CSV and plots) and *larger blocks* whose area is dominated by storage: the arbitrated memory of Lab R3 and the data cache of Lab R4. You will see that flip-flop storage grows linearly with capacity while the logic that reads it grows with the logarithm, find the path through the cache tag comparison that limits the clock, and argue, with numbers, why real chips use SRAM macros and what that does to the cache controller.

On the Synopsys cloud: `make sweep` runs Fusion Compiler and PrimeTime (`SYN_TOOL=fc`, `STA_TOOL=pt`, the defaults from `env.sh`) through exactly the same `synth`/`sta` targets as Labs P1 and P2. With the generic `class.db` every area is in class units and every period several times longer than the sky130 values of this text; keep those as the reference for orders of magnitude and compare *trends*: linear or logarithmic growth, which term dominates, which path is critical, ratios between configurations. That is also what the reviewer compares. SAED32 (`lib/README.md`) gives realistic 32 nm numbers and comes with SRAM macros for the macro comparison of Section 7.

By the end of this lab you should be able to:

1. Run synthesis and STA over a list of parameter sets with `make sweep` and turn the results into a CSV and plots.
2. Estimate the area of a storage-dominated block from bit counts and cell areas, and check the estimate against synthesis.
3. Break down the area of a block by function (data, tags, control, counters) using hierarchy-aware statistics.
4. Explain why the tag-compare/hit path is critical in almost every cache, and what designers do about it.
5. Compare flip-flop storage with SRAM macros and explain how a synchronous SRAM read changes a cache controller.
6. Write a short PPA report that supports a design decision.

Where this lab sits. It reuses `synth` and `sta` from Labs P1 and P2 unchanged and runs them on the blocks of Labs R3 and R4 (verified in Labs V2 and V3). The cache area numbers you produce are the area budget input of the cache recommendation memo in Lab T-AM1; the SDC style is extended to two clocks in Lab T-RTL2.

2 How Storage-Dominated Blocks Scale

Before running anything, build a model. For a block whose state is an array of N bits implemented in flip-flops:

$$A \approx N \cdot (a_{ff} + a_{we}) + W \cdot a_{mux}(D) + A_{ctrl}, \qquad t_{read} \approx t_{dec} + \lceil \log_4 D \rceil \cdot t_{mux4}$$

where a_{ff} is the area of one flip-flop, a_{we} the write-enable logic per bit (in our flow, a `mux2` feeding back Q), W the read width, $a_{mux}(D)$ the area of a D:1 multiplexer per output bit ($\approx (D-1)/3$ `mux4` cells), and A_{ctrl} the control logic that does not depend on D. With sky130 `hd` cells ($a_{ff} = 20.02$,

$a_{we} = 11.26$, $a_{mux4} = 21.27\,\mu\text{m}^2$) a stored bit costs about $31\,\mu\text{m}^2$ plus its share of the read mux, roughly 35–$40\,\mu\text{m}^2$ in total. Two conclusions follow, and this lab checks them:

- **Area is linear in capacity.** Doubling the depth doubles the area (minus the constant control part).
- **Delay is logarithmic in depth.** Each factor of four in depth adds one mux level (plus some buffering because the address bits fan out to every mux). Large flip-flop arrays are therefore an area problem first and a timing problem second.

Power follows area for leakage and follows activity for dynamic power: every flip-flop toggles its clock pin every cycle, so a large flop array burns clock power even when idle, unless clock gating is used. This lab does not run a power analysis; OpenSTA's `report_power` with default activity is an optional exploration.

On the Synopsys cloud: The model with `class.lib` cells: a_{ff} is `FD1` (7 units) or `FD2` with asynchronous clear (9), a_{we} a `MUX21H`-type cell feeding back Q, a_{mux} built from `MUX21H`/`MUX31L` and `AO`/`ND` gates. Read the unit areas from the `report_reference` of your netlist (`<top>_stat.rpt`) rather than from the `.lib`: Fusion Compiler chooses the cells. Power, optional: PrimePower in `pt_shell` (`set_app_var power_enable_analysis true`, `report_power`) with default switching activity, or with the activity of a GLS run (VCS SAIF/FSDB).

3 Parameter Sweeps

3.1 `syn/configs.txt`

Each IP lists the parameter sets of interest in `hw/ip/<ip>/syn/configs.txt`: one `PARAMS` value per line, `#` comments allowed.

```
# hw/ip/arb_mem/syn/configs.txt -- PARAMS sets for 'make sweep'
AW=2 DW=32
AW=4 DW=32
AW=6 DW=32
```

3.2 `sweep.sh` and the `sweep` Target

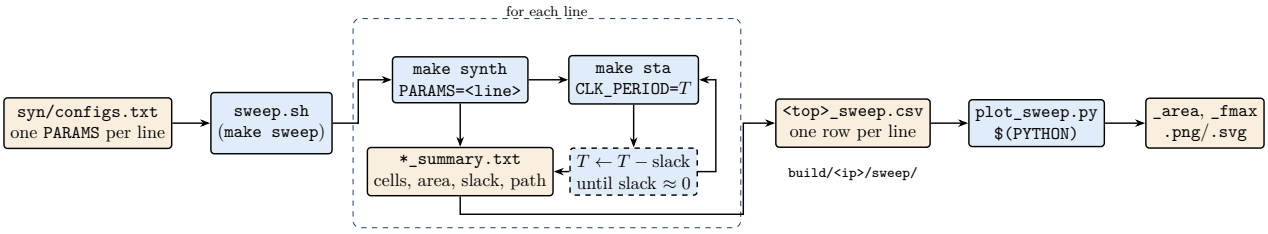


Figure 1: The sweep: for every parameter set, synthesis once and STA iterated to the minimum period; both write a small key/value summary, `sweep.sh` turns the summaries into one CSV row, and `plot_sweep.py` draws area and F_{max} per configuration.

Figure 1 shows the data flow; the listing below is the core of the script.

`flow/scripts/sweep.sh` calls `make synth sta` for each line, extracts the numbers from the reports and appends a CSV row. It does not know where the build directories are: it asks `make` with a `print-%` helper, so the configuration naming stays in `common.mk`.

```
#!/usr/bin/env bash
# flow/scripts/sweep.sh <configs.txt> <out.csv> <start_period_ns>  (Lab P3)
# Called by 'make sweep' from an IP directory; needs DUT_TOP in the environment.
set -euo pipefail
cfgs=$1; csv=$2; start=$3
echo "params,cells,flops,area_um2,tmin_ns,fmax_mhz,startpoint,endpoint" > "$csv"
grep -v -E '^\s*(#|$)' "$cfgs" | while read -r params; do
  d=$(make -s --no-print-directory print-BUILD_DIR PARAMS="$params")
  p=$start
  for i in 1 2 3 4; do                # fixed-point iteration: T <- T - slack
    make -s synth sta PARAMS="$params" CLK_PERIOD="$p" > /dev/null
    log=$d/sta/${DUT_TOP}_p${p}.log
    slack=$(awk '/^worst slack/ {print $NF; exit}' "$log")
    last=$p
    p=$(printf '%.3f' "$(echo "$p - $slack" | bc -l)")
  done
  stat=$d/syn/${DUT_TOP}_stat.rpt
  rpt=$d/sta/${DUT_TOP}_p${last}_setup.rpt
  cells=$(awk '/Number of cells/ {print $NF; exit}' "$stat")
  flops=$(awk '/sky130_fd_sc_hd__(df|edf|sdf)/ {s += $2} END {print s+0}' "$stat")
  area=$(awk '/Chip area/ {print $NF; exit}' "$stat")
  sp=$(awk '/^Startpoint:/ {print $2; exit}' "$rpt")
  ep=$(awk '/^Endpoint:/   {print $2; exit}' "$rpt")
  fmax=$(printf '%.1f' "$(echo "1000 / $p" | bc -l)")
  echo "\"$params\",$cells,$flops,$area,$p,$fmax,$sp,$ep" >> "$csv"
done
```

Why iterate? Our SDC scales the I/O delays with the period (20 %), so for a path that starts or ends at a port, $T - \text{slack}$ measured at 10 ns is not the minimum period: at 10 ns the input delay is 2 ns, at 2 ns it is only 0.4 ns. Repeating $T \leftarrow T - \text{slack}$ converges quickly (the error shrinks by the I/O fraction each step); for register-to-register paths the first step is already exact. Verify the final value with one more `make sta` at that period. The `awk` patterns match the classic `stat` format; adapt them if your Yosys prints a different layout.

The `sweep` target is added to `flow/mk/sta.mk`, together with the `print-%` helper; `flow/scripts/plot_sweep.py` turns the CSV into two PNG plots (area and F_{max} versus configuration) next to it.

```
# flow/mk/sta.mk (continued) -- parameter sweep (Lab P3)
CONFIGS   ?= syn/configs.txt
SWEEP_CSV := $(REPO_ROOT)/build/$(IP)/sweep/$(DUT_TOP)_sweep.csv

.PHONY: sweep
sweep:
        @mkdir -p $(dir $(SWEEP_CSV))
        DUT_TOP=$(DUT_TOP) $(REPO_ROOT)/flow/scripts/sweep.sh \
          $(CONFIGS) $(SWEEP_CSV) $(CLK_PERIOD)
        $(PYTHON) $(REPO_ROOT)/flow/scripts/plot_sweep.py $(SWEEP_CSV)
        @column -s, -t $(SWEEP_CSV)

print-%:
        @echo $($*)
```

```
# flow/scripts/plot_sweep.py (outline)
#   read the CSV with csv.DictReader; x = params labels
#   figure 1: bar chart of area_um2, annotated with flops
#   figure 2: line chart of fmax_mhz
#   save <csv-stem>_area.png and <csv-stem>_fmax.png next to the CSV
```

Call Python as `$(PYTHON)` (set by `env.sh` to a Python $\geq$ 3.8), never as `python3`: on RHEL 8 `python3` is 3.6.

On the Synopsys cloud: The `awk` patterns of the listing read Yosys `stat` and OpenSTA output. To keep `sweep.sh` independent of the tools, the repository's `syn.mk` and `sta.mk` write a key/value summary in every tool branch (`syn/<top>_summary.txt`: `cells flops latches area seq_area seq_fraction`; `sta/<top>_p<T>_summary.txt`: `slack_max tns slack_min startpoint endpoint path_type`), and `sweep.sh` reads only those: it runs unchanged with Fusion Compiler and PrimeTime. It iterates until $|\text{slack}| < 5$ ps, then verifies the final period with one more `make sta`; the CSV also records the slack of that verification run and the path type (`reg2reg`, `in2reg`, ...). The cloud's Python has no `matplotlib`: `plot_sweep.py` then writes the plots as SVG (and converts them to PNG with `rsvg-convert`). One sweep line costs one Fusion Compiler run (1–5 minutes for these blocks) plus 2–4 PrimeTime runs.

4 `arb_mem`: Memory Built from Flip-Flops

`arb_mem` has no SRAM macro: Yosys reads the unpacked array `mem[DEPTH]`, cannot map it to a memory cell of the library, and builds it from flip-flops, write-enable logic and a read multiplexer (log: `Replacing memory ... with list of registers` or `memory_map`). This is why `DEPTH` is small in Lab R3.

Interface ports. sv2v flattens the `mem_if` ports of the top module into ordinary ports, so the netlist has ports such as `m0_req_valid` and the SDC applies `all_inputs` / `all_outputs` to them unchanged. If your sv2v version rejects interface ports on the top module, synthesize a thin wrapper with flat ports instead (`DUT_TOP=arb_mem_syn_top`, file list in `syn/`) and record this in `doc/ppa.md`. GLS of these blocks is not required in this lab: the V2/V3 testbenches connect through interfaces.

On the Synopsys cloud: Fusion Compiler reads interface ports on the top module directly (no sv2v): `elaborate` splits each `mem_if` modport into ordinary ports, named `<port>_<signal>` (for example `m0_req_valid`), so the SDC with `all_inputs`/`all_outputs` applies unchanged and no wrapper is needed. Signals of the interface that the modport does not use still become ports: `mem_if`'s own `clk`/`rst_n` appear as unconnected inputs `m0_clk`, `m0_rst_n`, ...; only the real `clk` gets the clock. Check the port names in the netlist before you name a path in `ppa.md`.

Write `hw/ip/arb_mem/syn/arb_mem.sdc` (same template as Lab P2) and run `make sweep CLK_PERIOD=10.0`. An illustrative result:

```
params,cells,flops,area_um2,tmin_ns,fmax_mhz,startpoint,endpoint
"AW=2 DW=32",612,163,5284.6,1.888,529.7,m1_req_valid,_0412_
"AW=4 DW=32",2135,547,20316.0,2.637,379.2,m1_req_valid,_1630_
"AW=6 DW=32",8327,2083,79412.3,3.510,284.9,m1_req_valid,_6211_
```

(Illustrative and internally consistent: $163 = 4 \cdot 32 + 35$ control and response flip-flops; area per stored bit drops from about 41 to 39 μm^2 as the fixed control part is amortized. Your numbers will differ.) Fill in the table and answer the questions in `hw/ip/arb_mem/doc/ppa.md`.

AW	Bits stored	FFs	Area (μm^2)	μm^2/bit	T_{min} (ns)	Critical path
2	128					
4	512					
6	2048					

1. Compare the measured area with the model of Section 2. Which term dominates at `AW=6`?

2. By how much does T_{min} grow per `AW` step of 2? Which part of the critical path grows: the read mux, the address fan-out buffering, or the write-enable decode?
3. Extrapolate to `AW=10` (4 KB). Compare with the core area of a small RISC-V CPU (a few tenths of a mm^2 in this technology). What does this tell you?
4. The critical path starts at an input port. What does that say about the arbitration being combinational (`req_ready` depends on `req_valid`), and what would registering the request cost in latency?

5 `dcache`: Where the Area Goes

5.1 Model First

Per line the cache stores a valid bit, a dirty bit, a tag and `LINE_WORDS` data words (Lab R4). With the defaults (`ADDR_W=10`, `DW=32`, `LINE_WORDS=4`, `NUM_LINES=16`): `TAG_W=4`, so the tag, valid and dirty storage is $16 \cdot (4 + 2) = 96$ bits against $16 \cdot 4 \cdot 32 = 2048$ data bits. An illustrative breakdown from the model of Section 2:

Function	**Area (μm^2)**	**Share**
Data storage (2048 flip-flops with write enable)	64 061	69.9 %
Data read multiplexer (64:1, 32 bits)	14 294	15.6 %
Tag, valid and dirty storage (96 bits)	3 163	3.4 %
Tag read, compare and hit logic	700	0.8 %
Write path (data-in select, word enables)	800	0.9 %
Controller FSM, refill/evict counters, address registers	2 400	2.6 %
Performance counters (4×32 bits with incrementers)	4 739	5.2 %
Response registers and other	1 530	1.7 %
Total (illustrative)	91 687	100 %

Notice the performance counters: 128 flip-flops with reset cost more than all the tags. Real designs make such counters optional or share them; that is a PPA decision, too.

5.2 Measuring the Breakdown

`stat` on a flattened netlist gives one number. Two ways to split it:

1. **Memory wrappers and hierarchy (recommended).** Move the arrays into small modules in `hw/ip/dcache/rtl/` (for example `dcache_tag_ram` holding tag, valid and dirty, and `dcache_data_ram` holding the data words with their read mux), add them to `dcache.f`, and rerun the R4 smoke test and the V3 regression (the behavior must not change). Then `make synth SYN_FLATTEN=0` keeps the hierarchy and `stat` reports the area of every module. Industrial RTL always wraps memories like this, because the wrapper is exactly where an SRAM macro will be substituted later.
2. **Netlist names.** Without refactoring, count the flip-flops whose Q net carries each array's name in the flat netlist (`grep -c` on `.Q(` connections; check first how sv2v and Yosys named the array bits), multiply by the flip-flop plus write-enable area, and attribute the rest to logic. Less precise, but no RTL change.

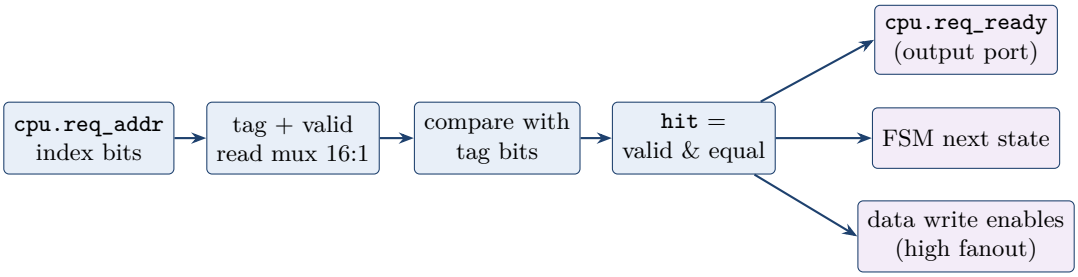
Test term — Smoke test and regression after a refactor. The R4 `smoke` test (a few directed accesses: hit, miss, refill) is the fastest proof that the refactored cache still works at all; the V3 regression (all tests, seeds and configurations of `dv/regress.list`, with its scoreboard and coverage) proves that the behavior did not change anywhere. A pure restructuring of the RTL must pass both before its area numbers mean anything.

On the Synopsys cloud: With Fusion Compiler, `SYN_FLATTEN=0` disables auto-ungrouping (`compile.flow.autoungroup false`) and `<top>_area.rpt` becomes `report_area -hierarchy`: one line per module instance with its combinational and sequential area. Fusion Compiler names elaborated modules after their parameter values (`<module>_<PARAM><value>...`, the way the top becomes `dcache_ADDR_W10_...` before the flow renames it). Without the wrappers, the netlist-name method works well with Fusion Compiler: every storage flip-flop is named `<array>_reg_<i>__<j>_`, so `grep -c 'data_q_reg'` on the netlist counts the data bits.

Report the breakdown for the default configuration and for `NUM_LINES=32`: which part dominates, and how does each part scale?

6 `dcache`: the Critical Path Through the Tag Compare

Run `make sta` for the cache (write `hw/ip/dcache/syn/dcache.sdc` from the template) and find the critical path. In almost every cache it looks like this:



Why this path is critical in almost every cache:

- **It is serial by nature.** The stored tag must be read (index decode and a wide mux) *before* it can be compared, and the comparison must finish *before* anything can be decided. Nothing in the chain can start early.
- **Its result controls everything.** `hit` drives the handshake, the FSM, and the write enables of the data array; the large fanout adds buffering delay at the very end of the path.
- **In our design it crosses the block boundary twice.** `req_ready` depends combinationally on the request address ("accept only on a hit", Lab R4), so the path is input-to-output and gets only T minus both I/O budgets (60 % of the cycle with our SDC).

What real designs do: read tags one stage earlier (address known early), pipeline the lookup (tag compare in stage 1, data in stage 2), compare a few tag bits early (partial tags, way prediction), or decouple `ready` from `hit` by accepting every request and stalling the response. Answer in `doc/ppa.md`: name the start and end of your critical path, map its cells to the RTL (P1 technique), give the share of its delay spent in the mux, the comparator and the fanout buffering, and say which of these techniques fits the `mem_if` protocol.

7 SRAM Macros versus Flip-Flops

A real cache stores its data, and usually its tags, in SRAM macros generated by a memory compiler for the technology.

	Flip-flop array (our flow)	**SRAM macro**
Area per bit	Flip-flop + enable + mux share	6-transistor bit cell: several times denser for arrays of a few Kbit and up
Fixed overhead	None	Decoders, sense amplifiers, control: flops win for very small arrays
Read timing	Asynchronous (combinational) read	Synchronous: address registered at the edge, data valid after the access time, i.e. in the next cycle
Ports	Any number (each costs a mux)	Fixed (1RW, 1RW1R, ...); more ports cost a lot
Reset	Possible (valid bits)	No reset: contents are random at power-up
Power	Every bit clocked every cycle	Only the accessed row switches
Test	Scan flip-flops	Memory BIST, repair rows

7.1 How a Synchronous Read Changes the Controller

With flip-flops, the tag is available combinationally in the same cycle as the address, which is what makes "accept only on a hit" and a read response exactly one cycle after acceptance possible. With a synchronous SRAM, data appears one cycle after the address is presented. Discuss the options in `hw/ip/dcache/doc/ppa.md`:

1. **Tags in flip-flops, data in SRAM.** The hit is still known combinationally; the data SRAM is read at the acceptance edge and its output is the response one cycle later. This matches the `mem_if` timing exactly, and the tags are a small fraction of the bits (96 of 2144). Refill and CPU writes now compete for the single SRAM port.
2. **Everything in SRAM, extra lookup cycle.** Present the index while `req_valid` is held, compare in the next cycle, then accept. Simple, but every access costs one more cycle: hit latency and throughput both suffer.
3. **Pipelined lookup with a changed protocol.** Accept every request, look up in the next stage, and stall or replay on a miss. Full throughput, but the response latency becomes variable, which `mem_if` does not allow; the protocol (and the V3 testbench expectations) would have to change.

In each case, also state what happens to the reset of valid bits (an SRAM cannot be reset in one cycle: flush by walking all indices after reset, or keep valid bits in flip-flops). **Task:** look up an OpenRAM-generated sky130 SRAM macro (the `sky130_sram_*` macros used by open-source tape-outs), read its size from the LEF or datasheet, and compare its area per bit with your flip-flop result for the same capacity. Explain the result, including the fixed overhead.

On the Synopsys cloud: The SAED32 kit on the cloud (`lib/README.md`) contains three SRAM macros generated by a memory compiler, with `.db`, NDM and GDS: 32×32, 64×32 and 2048×32 bits (names `sadcls0c4l1p32x32...`, `sadv1s0c4l1p64x32...`, `sadsls0c4l2p2048x32...`). Their area is the `area` of the cell in the `.db` (`get_attribute [get_lib_cells <lib>/<macro>] area` in `pt_shell`); the 32×32 macro, for example, is $23.36 \times 82.56\,\mu\text{m}$. Compare them with a SAED32 synthesis of your flip-flop array of the same size — both numbers in the same technology — instead of mixing sky130 and class units.

8 Cache Geometry Sweep: NUM_LINES and LINE_WORDS

Sweep the configurations below (`hw/ip/dcache/syn/configs.txt`, one line per row, e.g. `NUM_LINES=8 LINE_WORDS=4`). The bit counts follow from the R4 address-field formulas (`ADDR_W=10`, `DW=32`) and are given; the rest you measure.

Lines	Words/line	Capacity	TAG_W	Tag+V+D	Data bits	Area	T_{min}	Crit. path
8	4	32	5	56	1024			
16	4	64	4	96	2048			
32	4	128	3	160	4096			
32	2	64	4	192	2048			
8	8	64	4	48	2048			

1. For the three configurations of equal capacity (64 words), explain the area differences from the tag overhead and from the structure of the read mux. Which organization is cheapest?
2. Which configuration has the longest tag path, and why does `NUM_LINES` affect T_{min} more than `LINE_WORDS`?
3. Combine with your R4/V3 hit-rate measurements: which configuration would you choose for an area budget of 100 000 μm^2? (Lab T-AM1 repeats this decision with a model and realistic traces.)

8.1 Stretch: 2-Way Set-Associative versus Direct-Mapped

If you built the 2-way cache (R4 stretch, `WAYS=2`), add it to the sweep at equal capacity. Compare area (one more tag bit per line, a second comparator, LRU bits, way-select mux) and the critical path (the way multiplexer now sits *after* the two parallel compares). Relate the cost to the hit-rate gain on the `conflict` pattern (the R4/V3 access pattern that alternates between addresses mapping to the same index: every access misses in a direct-mapped cache and hits in a 2-way one).

9 The PPA Report

Each block gets a short, decision-oriented PPA section in `hw/ip/<ip>/doc/ppa.md`. Template:

```
## PPA summary (sky130_fd_sc_hd, tt_025C_1v80, Yosys <ver>, OpenSTA <ver>)
Reproduce: make sweep CLK_PERIOD=10.0     (commit <hash>)
| PARAMS | cells | FFs | area um2 | Tmin ns | Fmax MHz | critical path |
Area breakdown: <table or per-module stat>
Critical path: <start -> end, what it passes, % mux / logic / fanout>
Scaling: <what grows linearly, what logarithmically, measured vs model>
Limitations: no wires, ideal clock, typical corner, flop-based memory
Recommendation: <one paragraph: configuration and why>
```

On the Synopsys cloud: Name the real tools and library in the header line, for example `class.db (generic), Fusion Compiler X-2025.06-SP1, PrimeTime X-2025.06`, and state in `Limitations` that `class.db` is a training library (units, speed). The tool versions are in the first lines of `<top>_fc.log` and of the PrimeTime log.

The final review of Phase 3 is a short presentation (10 minutes, 5–8 slides): the scaling plots of `arb_mem`, the cache area breakdown, the critical path with your mapping to RTL, the SRAM recommendation, and one thing that surprised you.

10 Further Reading: Toward Place and Route

Everything in Phase 3 ends at a netlist with an ideal clock and no wires. The open-source OpenROAD flow (OpenROAD-flow-scripts, which has a `sky130hd` platform; OpenLane 2 wraps it) continues from exactly the Yosys netlist and the SDC you wrote:

1. **Floorplan:** die and core size from the cell area and a target utilization (typically 40–70 %), I/O pins, power grid, macro (SRAM) placement.
2. **Placement** of the standard cells, with `repair_design`: buffering of high-fanout nets (the fanout violations of Lab P2) and fixing of slew/capacitance limits.
3. **Clock-tree synthesis:** the ideal clock becomes a tree with real latency and skew; `set_clock_uncertainty` can be reduced to jitter only; hold violations appear and are fixed by inserting delay.
4. **Routing**, then **parasitic extraction** (SPEF) and sign-off STA with OpenSTA on the extracted netlist, at several corners.

Optional exploration (not graded): run OpenROAD-flow-scripts on `seq_mul` with your SDC and compare the post-route F_{max} and area with your P2 numbers.

On the Synopsys cloud: The Synopsys continuation is Fusion Compiler itself: with a physical library (SAED32 NDM and technology file instead of the bring-up library) the same tool runs floorplanning, `compile_fusion` (placement and optimization), `clock_opt` (clock tree) and `route_opt`, then StarRC extraction and PrimeTime sign-off on the SPEF. The Lynx flow installation on the cloud contains a complete SAED32 reference flow (`/global/apps/lynxnxt/`). Not part of the core.

11 Requirements / Tasks

1. Add `flow/scripts/sweep.sh`, `flow/scripts/plot_sweep.py` and the `sweep` and `print-%` targets to `flow/mk/sta.mk`. Nothing in them names a design.
2. Include `syn.mk` and `sta.mk` in the Makefiles of `arb_mem` and `dcache`; write their SDC files and `syn/configs.txt`. Zero latches in both.
3. `arb_mem`: sweep `AW=2/4/6`, fill in the table, compare with the model, answer the questions.
4. `dcache`: area breakdown for the default configuration and `NUM_LINES=32` (memory-wrapper refactor recommended; if done, V3 regression must still pass).
5. `dcache`: critical path analysis through the tag compare, mapped to RTL, with the share of mux, compare and fanout delay.
6. SRAM discussion: the three controller options, the reset of valid bits, and the OpenRAM macro comparison.
7. `dcache` geometry sweep with the questions answered.
8. Stretch: 2-way versus direct-mapped comparison.
9. PPA section in `doc/ppa.md` of both IPs, and the review presentation.

12 Repository and Flow

Path	Change in this lab
`flow/scripts/sweep.sh`	New: synth + sta over `configs.txt`, CSV
`flow/scripts/plot_sweep.py`	New: plots from the CSV
`flow/mk/sta.mk`	Extended: `sweep`, `print-%`, `CONFIGS`
`hw/ip/arb_mem/syn/arb_mem.sdc`, `.../syn/configs.txt`	New
`hw/ip/dcache/syn/dcache.sdc`, `.../syn/configs.txt`	New
`hw/ip/arb_mem/Makefile`, `hw/ip/dcache/Makefile`	Include `syn.mk`, `sta.mk`; set `SDC`
`hw/ip/dcache/rtl/dcache_tag_ram.sv`, `dcache_data_ram.sv`	Optional: memory wrappers (own commit, V3 regression in the message)
`hw/ip/<ip>/doc/ppa.md`	PPA section
`build/<ip>/sweep/`	Generated: CSV and plots

Target / variable	Function
`sweep`	For each line of `CONFIGS`: `synth` + iterated `sta`; writes `build/<ip>/sweep/<top>_sweep.csv` and plots
`CONFIGS`	Parameter-set list, default `syn/configs.txt`
`CLK_PERIOD`	Start period of the iteration (default 10.0)
`print-<VAR>`	Prints a make variable (used by `sweep.sh`)
`SYN_FLATTEN=0`	From Lab P1: keep the hierarchy for per-module `stat`
`<top>_<list>_sweep.csv`	With `CONFIGS` other than `syn/configs.txt`, the CSV name contains the list name (`dcache_configs_2way_sweep.csv`), so a stretch sweep never overwrites the main one

```
cd $REPO_ROOT/hw/ip/arb_mem
make sweep CLK_PERIOD=10.0
cd ../dcache
make synth SYN_FLATTEN=0                    # per-module area
make sta CLK_PERIOD=6.0
make sweep
make sweep CONFIGS=syn/configs_2way.txt     # stretch
```

On the Synopsys cloud: The same commands with Fusion Compiler and PrimeTime; with `class.db` start the iteration at `CLK_PERIOD=20` for `dcache` (the default 10 ns is far from its T_{min} in that library; the iteration converges from either side, but more slowly).

13 Deliverables

Merge request `lab/p3-ppa` (tag `p3-done`) containing:

- The sweep infrastructure, SDC files, `configs.txt` files and updated Makefiles.
- Optional memory-wrapper refactor of `dcache` as a separate commit, with the V3 regression result in its message.

- `hw/ip/arb_mem/doc/ppa.md`: PPA section with the sweep table, plots, model comparison and answers.
- `hw/ip/dcache/doc/ppa.md`: PPA section with the area breakdown, critical-path analysis, SRAM discussion and macro comparison, geometry sweep, (stretch) 2-way comparison, and a recommendation.
- The review presentation (PDF in `doc/reviews/`).
- READMEs of both IPs: `synth`, `sta`, `sweep` commands and headline numbers.

14 Assessment

Criterion	Weight
Sweep infrastructure: generic, reproducible, correct T_{min} iteration	15%
`arb_mem` scaling study and comparison with the model	15%
`dcache` area breakdown	15%
Tag-compare critical path: found, mapped to RTL, explained	15%
SRAM versus flip-flops: controller options and macro comparison	10%
`dcache` geometry sweep and questions	15%
PPA reports and review presentation	15%
Total	100%

The 2-way stretch adds up to 10 bonus points.

15 Review Questions

1. Estimate the area of a 32-word, 32-bit flip-flop memory in sky130 `hd` without running a tool. Which cells did you assume?
2. Why does the area of `arb_mem` grow linearly with `DEPTH` but its T_{min} only logarithmically?
3. Why is T − slack not the minimum period for a path that starts at an input port in our SDC, and how does `sweep.sh` handle it?
4. In the default cache, which costs more area: the tags or the performance counters? What would you do about it in a product?
5. Why is the path through the tag comparator usually critical? Name two techniques that shorten it.
6. Why does a synchronous SRAM read break “accept only on a hit”? Which controller option keeps the `mem_if` protocol unchanged, and what does it cost?
7. Why can a cache not reset its SRAM valid bits in one cycle, and what are the alternatives?
8. For equal capacity, why do more, shorter lines cost more area?
9. What does the synthesized memory consist of, and why would a real chip use an SRAM macro instead? When would flip-flops still be the right choice?
10. Which three effects missing from our flow would change your numbers most after place and route?

16 References

- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: chapter on memory systems.
- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*: memory hierarchy chapter and the appendix on cache optimizations (hit time techniques).
- Weste and Harris, *CMOS VLSI Design*: chapter on array subsystems (SRAM organization, decoders, sense amplifiers).
- OpenRAM: open-source memory compiler and its sky130 macros.
- OpenROAD-flow-scripts and OpenROAD documentation: floorplan, placement, CTS, routing, `repair_design`, `repair_timing`.
- Yosys manual: `memory` passes and hierarchy handling in `stat`; OpenSTA documentation: `report_power`.